Problem 1

(a)

i.

	HBM	On-chip SRAM
Location	Off-chip DRAM	On the GPU die
Capacity	40-80 GB	About 20 MB across SMs
Bandwidth	About 1.5-2.0 TB/s	About 19 TB/s in aggregate

ii. Standard attention transfers intermediates as follows:

1. Load Q, K from HBM, compute $S = QK^\top$, and write S to HBM.
2. Read S, compute $P = \mathrm{softmax}(S)$, and write P to HBM.
3. Read P, V, compute $O = PV$, and write O to HBM.

The two $N \times N$ intermediates S and P require approximately $4N^2$ element transfers. But Softmax performs little arithmetic per byte transferred. Therefore, this quadratic traffic can make HBM bandwidth the bottleneck despite high GPU FLOP throughput.

(b)

Let $m_k = m(x^{(k)})$, $\ell_k = \ell(x^{(k)})$ for $k = 1, 2$, and $m = \max(m_1, m_2)$.

i.

$$\begin{aligned}
\ell(x) &= \sum_{r=1}^{B} e^{x_r^{(1)} - m} + \sum_{r=1}^{B} e^{x_r^{(2)} - m} \\
&= \sum_{r=1}^{B} e^{(x_r^{(1)} - m_1) + (m_1 - m)} + \sum_{r=1}^{B} e^{(x_r^{(2)} - m_2) + (m_2 - m)} \\
&= e^{m_1 - m} \sum_{r=1}^{B} e^{x_r^{(1)} - m_1} + e^{m_2 - m} \sum_{r=1}^{B} e^{x_r^{(2)} - m_2} \\
&= e^{m_1 - m} \ell_1 + e^{m_2 - m} \ell_2.
\end{aligned} \tag{1}$$

ii.

$$\begin{aligned}
&\because \quad f(x) = \left[e^{m_1 - m} f(x^{(1)}),\ e^{m_2 - m} f(x^{(2)}) \right], \\
&\therefore \quad \frac{f(x)_i}{\ell(x)} = \frac{e^{x_i - m}}{\sum_{j=1}^{2B} e^{x_j - m}} = \frac{e^{x_i} e^{-m}}{\sum_{j=1}^{2B} e^{x_j} e^{-m}} \\
&\qquad\qquad = \frac{e^{x_i} e^{-m}}{e^{-m} \sum_{j=1}^{2B} e^{x_j}} = \frac{e^{x_i}}{\sum_{j=1}^{2B} e^{x_j}} \\
&\qquad\qquad = \mathrm{softmax}(x)_i, \qquad i = 1, \ldots, 2B,
\end{aligned} \tag{2}$$

(c)

i. Standard backpropagation saves the attention probabilities $P \in \mathbb{R}^{N \times N}$ and typically materializes gradients with respect to P and S, also of size $N \times N$. These tensors require $O(N^2)$ memory.

ii. FlashAttention only saves Q, K, V, O and per-row normalization statistics. During backward computation, it reconstructs score and probability tiles in SRAM. Each tile is used to accumulate gradients and then discarded. This requires $O(Nd_k)$ storage plus $O(N)$ statistics and fixed tile workspace, hence $O(N)$ memory for fixed head dimensions and head count.

iii. Recomputing score tiles adds FLOPs but avoids reading and writing quadratic intermediates in HBM. Fast GPU matrix arithmetic and SRAM reuse can cost less time than those HBM transfers, making the backward pass faster in the bandwidth-bound regime.

Problem 2

(a)

i.

Architecture	KV heads	Bytes per individual token	Bytes added per step for batch B
MHA	n_h	$4d_{\text{model}}$	$4Bn_hd_h$
MQA	1	$4d_h$	$4Bd_h$
GQA	G	$4Gd_h$	$4BGd_h$

ii. MQA and GQA reduce KV cache reads from HBM by sharing Keys and Values across Query heads. However, each Query head still computes its own attention, so the main FLOPs remain nearly unchanged. Since decoding is memory-bandwidth-bound, reducing memory I/O significantly lowers latency.

(b)

i. By absorbing the key up-projection into the Query, attention can be computed directly using the cached compressed latent vectors, without reconstructing or storing full-dimensional Keys. This preserves the original attention scores while reducing memory overhead.

$$\begin{aligned} Q_{t,i}K_{j,i}^\top &= Q_{t,i}\big(c_j^{KV}W_i^{UK}\big)^\top \\ &= Q_{t,i}(W_i^{UK})^\top (c_j^{KV})^\top \\ &= \widetilde{Q}_{t,i}(c_j^{KV})^\top, \\ \widetilde{Q}_{t,i} &= Q_{t,i}(W_i^{UK})^\top \in \mathbb{R}^{1\times d_c}. \end{aligned} \tag{3}$$

ii.

$$S_{t,j,i} = \frac{\widetilde{Q}_{t,i}(c_j^{KV})^\top}{\sqrt{d_h}}, \qquad j \le t. \tag{4}$$